Proof of Weights: Per-Layer Activation Profiles for Attributing Weight Tampering in Verifiable LLM Inference

Author Name
Indian Institute of Technology Delhi
ms1240669@iitd.ac.in

Abstract

Decentralised inference markets pay GPU providers to run a named open-weight model, but a provider can quietly serve a quantised or fine-tuned copy instead. TOPLOC (Ong et al., ICML 2025) commits to the top-k entries of the last hidden state and verifies them with a single prefill, but its authors note that small fine-tuning updates are harder to detect than model swaps and leave them to future work. We study this gap on Qwen2.5-1.5B-Instruct with four tampered variants: 8-bit and 4-bit (NF4) quantisation, a merged LoRA adapter, and a 300-step full fine-tune. First, a reproduction: with thresholds calibrated to a 2% held-out false-positive rate (FPR) on honest traffic that varies the attention kernel and batch shape, the last-hidden-state check detects 0% of quantised sequences and 55–60% of fine-tuned ones; the published thresholds yield a 23% FPR on the same honest traffic. Second, we extend the comparison from one layer to all 28 decoder layers. A logistic-regression classifier over a 112-dimensional per-layer profile separates honest traffic from the four tamper classes with 99.3% balanced accuracy under prompt-grouped cross-validation, and detects 90–100% of tampered sequences at a 1% FPR. Restricting the same classifier to the last layer drops balanced accuracy to 78.0%; the first half of the network alone reaches 98.6%. The result holds when both models read identical text (99.1%), so it reflects the weights rather than the generated tokens. Third, the profile does not generalise to an honest configuration that is absent from training: an unseen eager-attention kernel is flagged on 100% of prompts. Finally, we attach the classifier to a staked optimistic-challenge contract that records the attributed tamper class on chain, and we measure serving cost on an A10G, where both quantised modes decode *slower* than bf16, so a time-based stake bound gives no constraint in this setting.

1 Introduction

Open-weight language models are increasingly served by third parties: commercial APIs, decentralised compute networks, and peer-to-peer inference markets. The client names a model and pays per token, but has no direct way to see which weights produced the response. A provider can save memory or compute by quantising the model, or can serve a fine-tuned copy, and the text it returns will usually look fluent. Gao et al. [6] found that 11 of 31 commercial endpoints for four Llama models served output distributions that differ from the reference weights.

Cryptographic proofs of inference [7, 13] give strong guarantees, but they remain orders of magnitude too slow for serving. Trusted execution environments [1, 17] move trust to hardware vendors. Black-box statistical tests on output text [6] need many samples per prompt and lose power against small changes [1]. TOPLOC [11] occupies a middle ground: the provider commits to the top-k values of the last hidden state every 32 generated tokens, and a validator recomputes them with a single prefill pass. The check tolerates honest numerical noise from different GPUs

and kernels by thresholding exponent and mantissa disagreements, and the original paper reports perfect separation of model, prompt and precision changes. The TOPLOC authors also observe that detecting small gradient updates in fine-tuned models is harder than detecting a model swap, and they leave it open.

This paper asks two questions about that open case. (1) Does the last-hidden-state check detect quantisation and light fine-tuning once its thresholds are set to tolerate realistic honest noise? (2) If not, does the same top-k comparison, applied at every decoder layer, both detect the modification and *identify* which one occurred? Attribution matters for enforcement: a staking protocol that slashes providers can record what was detected, and the record distinguishes a cost-saving quantisation from a behaviour-changing fine-tune.

Contributions.

- **A calibrated reproduction with a negative result** (Section 6.1). We calibrate TOPLOC's three thresholds on 100 honest prompts that vary attention kernel and batch shape, and validate on a disjoint 100 (2% FPR). Under these thresholds the last-hidden check misses all 8-bit and 4-bit sequences, and catches 59.5% (LoRA) and 54.5% (full fine-tune) of the fine-tuned ones. The published thresholds produce a 23% FPR on the same honest prompts. We trace the failure to one noisy honest axis, the eager attention kernel.
- **Per-layer attribution** (Sections 6.2 and 6.3). A linear classifier on per-layer top-k disagreement profiles reaches 99.3% five-class balanced accuracy and 90–100% detection at 1% FPR. Ablations show that the gain comes from depth: the last layer alone gives 78.0%, the first 14 layers give 98.6%. Identical-text controls and pre-EOS controls rule out the generated text as the main signal.
- **A generalisation failure, measured** (Section 6.5). When one honest kernel configuration is withheld from training, the classifier flags all 200 of its prompts as tampered. Per-layer profiles are only as reliable as the honest configurations they were trained on.
- **Enforcement and cost** (Sections 4.4 and 6.6). We implement an optimistic challenge contract with a stake, a challenge bond and a typed tamper class, and run an end-to-end slash. We measure serving cost and find that on our A10G setup the quantised models decode 1.3–4.9× slower than bf16, so a time-saving argument for a large stake does not apply there.

All result files, the feature-extraction code shared by the offline and live paths, and the contract are released with the paper.

2 Background and related work

TOPLOC. For a sequence of generated tokens, TOPLOC [11] records the last hidden state at one position per block of 32 decode steps, keeps the k=128 entries of largest magnitude, and encodes their indices and values as a polynomial commitment of a few hundred bytes. A validator runs one prefill over the prompt and the claimed output, extracts the same positions, and compares. Because bf16 arithmetic differs across kernels and hardware, the comparison is tolerant: it counts top-k index mismatches, exponent mismatches between corresponding values, and the mean and median absolute mantissa difference, and accepts if each statistic is below a threshold. The published thresholds are T_{exp}=38, T_{mean}=10, T_{median}=8. The check is locality-sensitive in the sense of [10]: small honest perturbations map to small disagreements.

Verifiable and auditable inference. zkLLM [13] produces zero-knowledge proofs for 13B-parameter models in about 15 minutes per inference. SafetyNets [7] and Slalom [17] use interactive

proofs and trusted hardware. Cai et al. [1] show that text-only tests are query-hungry and that log-probability tests are confounded by inference nondeterminism, and argue for TEEs. SVIP [14] trains a proxy task on returned hidden states to detect substitution with a smaller model; it targets model swaps rather than quantisation or fine-tuning of the same model. Model equality testing [6] frames the problem as a two-sample test on output text. Our work stays within TOPLOC's activation-commitment model and asks how much more it can say about *which* modification occurred.

Optimistic verification. Optimistic rollups and Truebit [16] accept results by default and resolve disputes on challenge; opML [2] applies the pattern to ML inference with an interactive fraud-proof game. TOPLOC is protocol-agnostic and does not specify a stake or slash. We implement the simplest optimistic layer, with a single trusted validator, to show what an attribution label adds to the on-chain record.

Honest nondeterminism. LLM inference is not bitwise reproducible across batch sizes, padding and kernels [8]. Any activation-based check must tolerate this noise. Our results show that its size depends strongly on the source: switching the attention kernel produces far larger disagreements than changing the batch shape (Table 3).

3 Setting and threat model

A *provider* registers a commitment to a model (here, a Hugging Face repository and pinned revision), posts a stake, and serves requests. For each response it submits a proof hash on chain and stores the proof off chain. A *validator* samples submitted proofs, recomputes activations with the committed weights, and resolves challenges. *Challengers* post a bond to dispute a proof within a fixed window.

Adversary. The provider serves the requested prompt with weights $\tilde{\theta} \neq \theta$, where $\tilde{\theta}$ is a quantised or fine-tuned version of the committed weights θ, and produces the proof honestly from $\tilde{\theta}$'s activations. We consider four modifications (Table 1). We do not consider a provider that forges activations, serves a different prompt, or uses speculative decoding, which TOPLOC already identifies as undetectable by its check.

Honest variation. An honest provider runs θ but may use a different attention kernel, batch size or padding than the validator. A useful check must accept such traffic. We model this with three honest perturbation axes (Section 5).

Trust assumptions. The validator is trusted to run θ and to report honestly. The contract enforces the stake, the dispute window and the payout, but not the correctness of the verdict.

4 Method

4.1 Last-hidden-state check (reproduction)

We use the reference implementation (`toploc` 0.1.6) unchanged: k=128, one proof per 32 decode tokens, eight proofs per 256-token generation. For each sequence we take the maximum over its eight proofs of the three statistics (exponent mismatches, mean and median mantissa difference) and flag the sequence if any maximum exceeds its threshold. Proofs are built from CPU bf16 copies of the activations; the library's polynomial construction failed on CUDA tensors in our environment.

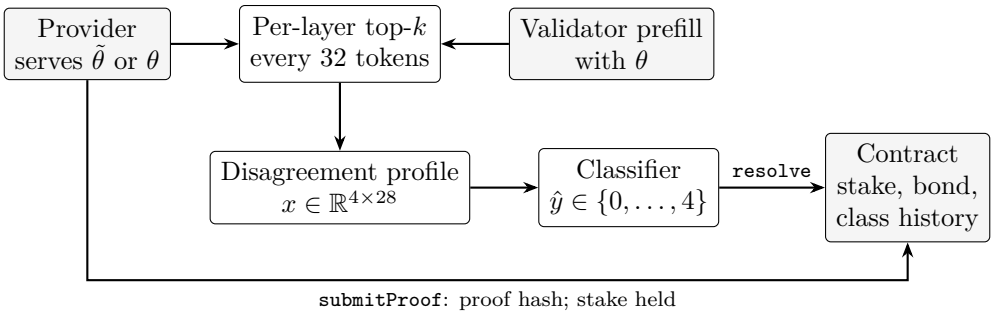


Figure 1: System overview. The provider's per-layer top-k activations are compared with the validator's prefill under the committed weights. The resulting disagreement profile is classified, and an upheld challenge slashes the stake and records the class on chain.

Calibration. Thresholds are the 99th percentiles of the three statistics over honest sequences from prompts 0–99, pooling the three honest axes. We report the realised FPR on prompts 100–199, which were not used for calibration, and require it to be below 3%. We also report the FPR of the published thresholds on the same held-out prompts.

4.2 Per-layer disagreement profiles

Let $h_t^{(\ell)} \in \mathbb{R}^d$ be the output of decoder layer $\ell \in \{0, \ldots, L-1\}$ at position t, with L=28 and d=1536. At each checkpoint position $t_c = |\text{prompt}| + 32c$, $c \in \{0, \ldots, 7\}$, we record the indices $I_c^{(\ell)}$ and values $v_c^{(\ell)}$ of the k=128 largest-magnitude entries of $h_{t_c}^{(\ell)}$ (sorted by magnitude). We do this for the model under test and for the reference model θ on the same token sequence, both with a teacher-forced prefill. For each (c, ℓ) we compute four statistics, using the same definitions as TOPLOC:

$$f_{\text{idx}} = 1 - \tfrac{1}{k}\,|\tilde{I} \cap I|, \qquad f_{\text{exp}} = \textstyle\sum_i \mathbb{1}[\exp(\tilde{v}_i) \neq \exp(v_i)],$$
$$f_{\text{mean}} = \text{mean}_i |\text{mant}(\tilde{v}_i) - \text{mant}(v_i)|, \qquad f_{\text{med}} = \text{median}_i |\text{mant}(\tilde{v}_i) - \text{mant}(v_i)|,$$

where exp and mant are the 8-bit exponent and 23-bit mantissa fields of the float32 representation, and i runs over rank positions. The profile of a sequence averages over checkpoints and stacks the layers: $x \in \mathbb{R}^{4\times 28}$, a 112-dimensional vector.

Matched and shared text. A tampered model generates different text from the reference. In the *matched* configuration, both models prefill the tampered model's own output, which is what a validator would see in deployment. In the *shared* configuration, both prefill the reference model's output for the same prompt, so any difference comes from the weights and not from the tokens.

4.3 Classifier and evaluation protocol

We fixed the following choices before running the experiment. The primary model is multinomial logistic regression with balanced class weights (seed 20260923); k-nearest neighbours (k=5, uniform weights) is a secondary baseline. Features are z-scored with statistics from the training fold only. Evaluation uses 5-fold `GroupKFold` grouped by prompt, so no prompt appears in both training and test data under any class. We report balanced accuracy, per-class F1 and confusion matrices from out-of-fold predictions.

For binary detection we score each sequence by $1 - P(\text{clean})$ from the out-of-fold five-class model and report ROC curves and the true-positive rate at 1% FPR, computed over all honest sequences.

Table 1: Tampered variants of the committed model. All four produce coherent text on a fixed sanity prompt.

Class	Variant	Construction	Weights changed
1	8-bit	bitsandbytes LLM.int8() [3], outlier threshold 6.0	all linear layers, quantised
2	4-bit	bitsandbytes NF4 [4], bf16 compute, no double quant.	all linear layers, quantised
3	LoRA	r=16, α=32, Q/K/V, lr 2×10^{-4}, 300 steps, merged [9]	attention Q/K/V
4	Fine-tune	full SFT, lr 10^{-5}, 8-bit AdamW, 300 steps	all

Two treatments of the honest class are reported. In the *pooled* treatment, which we fixed before the run, each prompt's three honest profiles (one per perturbation axis) are averaged into one clean vector, giving 200 vectors per class. In the *separate* treatment, each honest axis contributes its own 200 vectors (600 honest, 800 tampered). A deployed validator sees one honest configuration per request, so we treat the separate treatment as the main result.

4.4 Enforcement contract

The `ProofOfWeights` contract (Solidity, 142 lines) stores, per provider, a model hash, a stake in an ERC-20 token, a slashed flag and a list of `uint8` tamper classes; and, per proof, a prompt hash, a proof hash, an off-chain blob URI and a timestamp. Anyone may challenge a proof within a two-hour window by posting a bond. Only the validator address may call `resolveChallenge(id, upheld, class)`. An upheld challenge sets the stake to zero, pays half to the challenger and half to the validator, refunds the bond and appends the class; a rejected challenge forfeits the bond to the validator. Class identifiers are 0 clean, 1 8-bit, 2 4-bit, 3 LoRA, 4 full fine-tune, and 255 for an unattributed binary verdict. Seven Foundry tests cover honest providers, typed and binary slashes, false challenges, the dispute window, double slashing and validator-only resolution.

Stake bound. If a validator samples a fraction p of proofs and an upheld challenge slashes stake S, a provider that saves Δ per request by tampering is deterred when $\Delta < pS$ (one proof per epoch). We measure Δ as the difference in GPU time per request, converted at the on-demand hourly price of the instance.

5 Experimental setup

Model and data. We use Qwen2.5-1.5B-Instruct [12] (revision `989aa79`, 28 layers, hidden size 1536) in bf16. Evaluation prompts are 200 long-answer prompts sampled from UltraChat-200k [5]. Each generation is greedy, 256 new tokens long, with EOS suppressed so that every sequence has eight checkpoints; the natural EOS position is stored. In a 20-prompt length probe without suppression, 19 prompts ran to the 256-token cap. Fine-tuning data is a 2,400-row slice of Alpaca [15], disjoint from the evaluation prompts.

Tampered variants. Table 1 lists the four variants. Both trained variants use 300 optimiser steps with micro-batch 1, gradient accumulation 8, maximum length 512, and gradient checkpointing; each sees the same 2,400 Alpaca rows, in a different order (seeds 202609231 and 202609232). TF32 is disabled throughout.

Table 2: Last-hidden-state check with calibrated thresholds (2% held-out FPR), 200 prompts per variant. Statistics are medians over sequences of the per-sequence maximum over eight proofs.

Variant	Detected	Exp. mismatches	Mantissa mean	Mantissa median
8-bit	0.0%	9.0	3.30	3.0
4-bit	0.0%	20.0	9.74	8.0
LoRA	59.5%	43.5	24.93	21.0
Fine-tune	54.5%	41.0	23.69	19.5
Threshold	—	46.02	25.80	20.0

Table 3: Honest noise on the last-hidden statistics, by perturbation axis (mean over 200 prompts of the per-sequence maximum). The eager kernel is the dominant source of honest disagreement.

Honest axis	Exp. mismatches	Mantissa mean
Eager attention, batch 1	25.55	14.22
SDPA, batch 4	4.35	1.35
SDPA, batch 4, +17 padding	4.31	1.37

Honest perturbation axes. The reference run uses SDPA attention at batch size 1. Honest traffic is the reference weights under one of three changes: *eager* attention at batch 1; SDPA at batch 4; and SDPA at batch 4 with 17 extra padding tokens.

Hardware. All main experiments run on one NVIDIA A10G (24 GB, AWS `g5.xlarge`) with PyTorch 2.11 and CUDA 12.8. A cross-hardware check uses an NVIDIA L4 (compute capability 8.9, `g6.xlarge`).

6 Results

6.1 The last-hidden check misses quantisation and half of fine-tuning

Calibrated on prompts 0–99, the thresholds are T_{exp}=46.02, T_{mean}=25.80, T_{median}=20.0, and the realised FPR on prompts 100–199 is 2%. The published thresholds $\{38, 10, 8\}$ give a 23% FPR on the same held-out prompts, so we do not use them. Table 2 shows detection on all 200 prompts: neither quantised variant is ever flagged, and the two fine-tuned variants are flagged on just over half of the prompts. Every variant falls short of the 95% detection rate we set in advance as the reproduction criterion.

Diagnosis. The thresholds are set by the noisiest honest axis. Table 3 shows that the eager attention kernel produces exponent disagreements roughly six times larger than a change of batch shape, and larger than those of the 8-bit and 4-bit variants. Because calibration pools the three axes, the 99th percentile covers the eager tail, and any tamper whose last-layer footprint is smaller than eager-kernel noise cannot be detected by a threshold on these statistics. Thresholds calibrated without the eager axis would be tighter, but they would also reject honest providers who use a different attention kernel. We did not re-run the check with such thresholds and do not report a number for them.

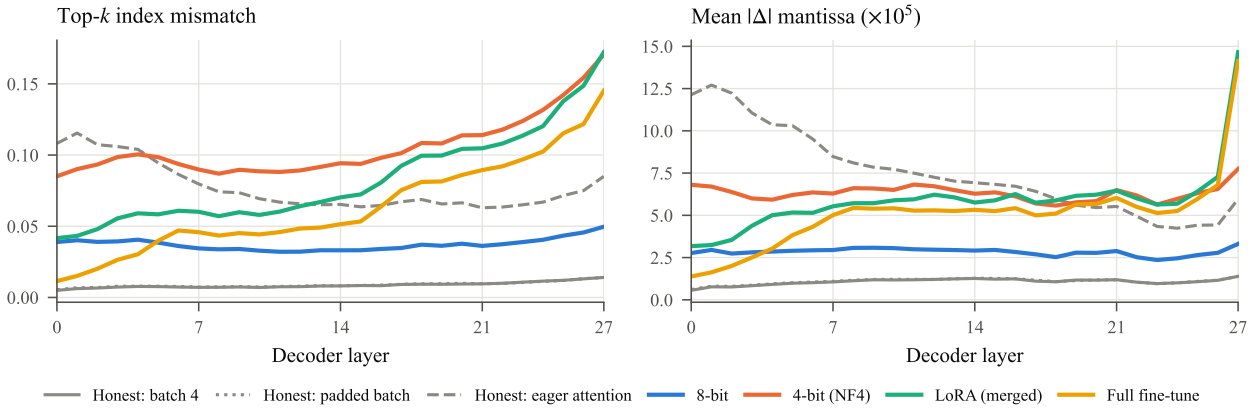


Figure 2: Mean per-layer disagreement with the reference prefill on matched text (200 prompts, eight checkpoints). Grey: honest axes. Colour: tampered variants. Exponent mismatch and median mantissa difference show the same pattern.

Table 4: Five-class attribution, logistic regression, 5-fold prompt-grouped cross-validation, out-of-fold predictions. Rows: true source. Columns: predicted class. Separate honest axes, matched text. Balanced accuracy 99.3%.

	Clean	8-bit	4-bit	LoRA	Fine-tune
Honest: eager attention	**187**	8	5	0	0
Honest: batch 4	**199**	0	0	0	1
Honest: batch 4 + padding	**199**	0	0	0	1
8-bit	1	**199**	0	0	0
4-bit	0	0	**200**	0	0
LoRA	0	0	1	**199**	0
Fine-tune	0	0	0	0	**200**

6.2 Per-layer profiles attribute the modification

Figure 2 shows mean per-layer disagreement by source. The two batch-shape axes stay near zero at every layer. Eager attention is large in early layers and decays with depth. 8-bit quantisation is small and flat; 4-bit is larger and roughly flat. The two trained variants start near zero, grow with depth, and spike at the final layer. Each source therefore has a distinct shape, and the eager kernel, which is indistinguishable from quantisation at the last layer, differs from it in the early layers.

Table 4 gives the out-of-fold confusion matrix with honest axes kept separate. Balanced accuracy is 99.3%. All 200 fine-tuned sequences are attributed correctly, and only one tampered sequence (8-bit) is called clean. The errors are concentrated in honest eager-attention traffic: 13 of 200 prompts (6.5%) are labelled as 8-bit or 4-bit, which is consistent with the profile overlap in Figure 2. The two batch-shape axes have a 0.5% FPR each.

The pooled treatment that we fixed before the run gives 99.5% (95% CI over folds 99.1–99.9%), with 2 of 200 clean vectors misclassified and all 200 fine-tuned sequences correct; k-NN gives 98.8%. Averaging the three honest axes into one vector hides the eager-kernel errors, which is why we lead with the separate treatment.

Detection at a fixed FPR. Figures 3 and 4 recast the classifier as a detector. At 1% FPR over all 600 honest sequences, it detects 90.0% of 8-bit sequences and 100% of the other three

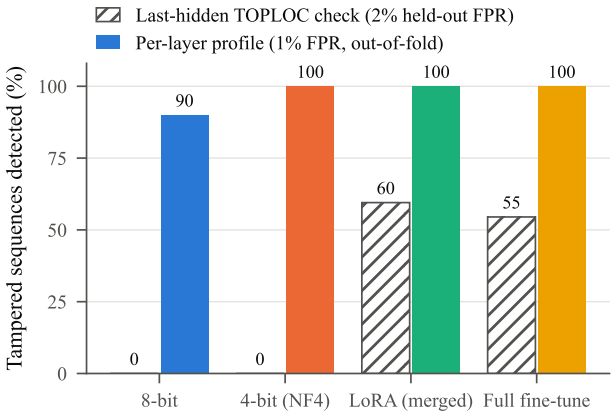


Figure 3: Detection rate by variant. Hatched: last-hidden TOPLOC check with calibrated thresholds. Solid: per-layer classifier at 1% FPR (out-of-fold).

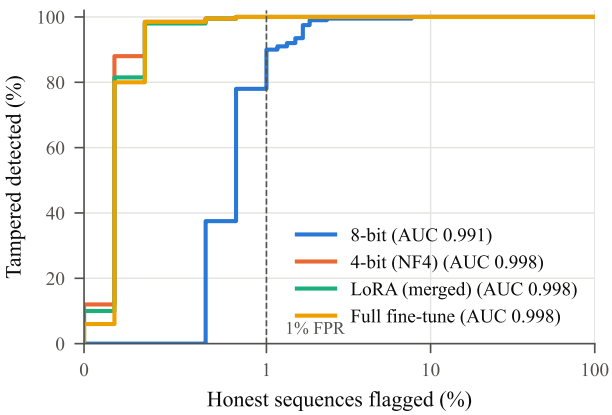


Figure 4: ROC of the per-layer score $1 - P(\text{clean})$, honest sequences from all three axes versus each variant. Log-scaled FPR axis.

variants; AUC is 0.991 for 8-bit and 0.998 for the rest. This comparison with the last-hidden check is not exactly like-for-like. The last-hidden check compares a provider's *decode* activations with the validator's prefill, which adds decode-versus-prefill noise, whereas both sides of the per-layer profile are prefills. The FPR budgets also differ (2% versus 1%). Section 6.3 isolates the effect of depth with everything else held fixed.

6.3 Depth, not the feature set, carries the signal

Figure 5 retrains the same classifier on subsets of the profile, with the split, seed and honest treatment unchanged.

- **Layers.** The last layer alone, the information available to the original TOPLOC check, reaches 78.0% balanced accuracy and flags 74.5% of honest eager traffic; 27% of 8-bit sequences are called clean. Adding every other layer raises accuracy to 99.3%. The first 14 layers alone reach 98.6%, the last 14 reach 91.6%, and dropping the last layer costs only 0.4 points. The early layers are where honest kernel noise and quantisation separate (Figure 2).
- **Features.** Mantissa statistics alone give 98.5%; index mismatch alone and exponent mismatch alone give 93.8% and 93.5%.
- **Checkpoints.** One checkpoint (the first 32 tokens) gives 90.9%; two give 94.7%; four give 97.6%; eight give 99.3%. Most of the gain comes from reducing honest eager-kernel false positives, from 36.0% to 6.5%.

6.4 The signal comes from the weights, not the text

A tampered model writes different text, and a classifier could in principle key on that. Two controls argue against this. With *shared* text, where both models prefill the reference output, balanced accuracy is 99.1% (separate axes) and 99.8% (pooled), against 99.3% and 99.5% on matched text. Restricting to checkpoints before the natural EOS position, so that no position falls in suppressed-EOS continuation, gives 98.8% for the pooled treatment on both matched and shared text. Table 6 in the appendix lists all configurations.

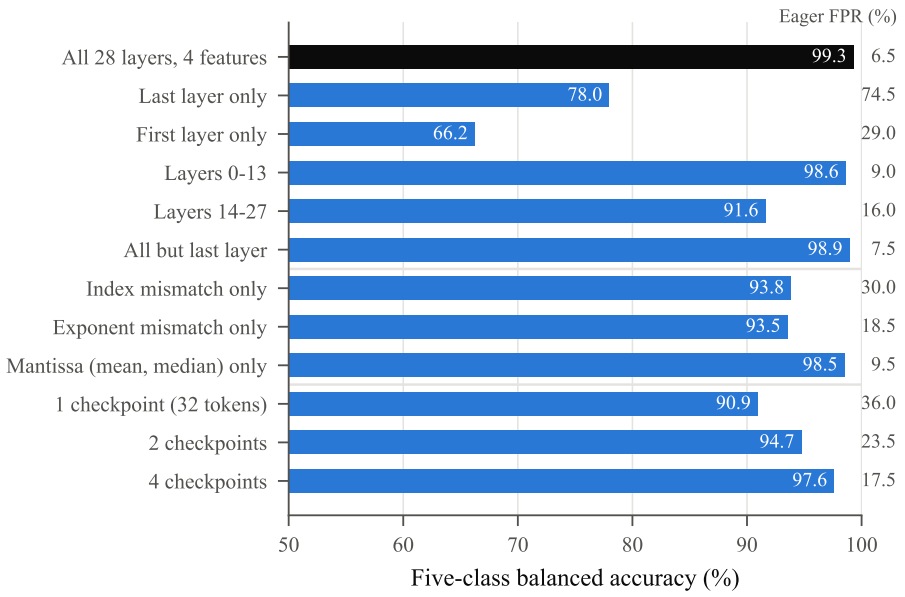


Figure 5: Ablations of the per-layer profile (matched text, separate honest axes). Bars: five-class balanced accuracy. Right column: share of honest eager-attention prompts flagged as tampered.

6.5 Generalisation to unseen honest configurations

The honest class in Table 4 includes the eager kernel at training time. To test robustness to an honest setup the validator has not seen, we withhold one axis from training and score it at test time (still grouped by prompt). Withholding either batch-shape axis is harmless: the withheld axis is flagged on 0.5% of prompts. Withholding eager attention is not: all 200 eager prompts are flagged, 182 as 4-bit and 18 as 8-bit. The classifier learns the honest configurations it is shown and does not extrapolate from batch-shape noise to a kernel change. A deployment would need to enumerate the kernels it accepts, or require providers to declare their kernel so the validator can match it.

Cross-hardware. We generated 50 honest sequences on an NVIDIA L4. The L4's own decode proofs, verified by an L4 prefill with the A10G-calibrated thresholds, were all accepted (exponent mismatches 2–9, median 5). Comparing L4 prefill per-layer top-k with the stored A10G reference on the same tokens, the A10G-trained classifier labelled all 50 as clean. Two caveats apply: 50 sequences is a small sample, and we did not verify A10G proofs against L4 activations, since the proof bytes were not transferred between instances.

6.6 Serving cost and the stake bound

Table 5 reports serving cost on the A10G at batch size 1 with greedy decoding (three timed 256-token requests after one warm-up). Both quantised modes are slower than bf16: 8-bit by 4.9× and 4-bit by 1.3×. The saving Δ is therefore negative (-25.7 s and -2.0 s per request, or $-\$0.0072$ and $-\$0.0006$ at \$1.006 per hour), and the bound $\Delta < pS$ holds for any $S \geq 0$ at sampling rates of 1%, 5% and 10%. The quantised models do use 39–56% less peak memory, which a provider could exploit by packing more replicas or larger batches per GPU. A memory-based cost model, or optimised low-bit kernels at higher batch sizes, may yield a positive Δ; we did not measure these. Validator cost is small: one prefill takes 0.046 s on average, 0.69% of a bf16 generation.

Table 5: Serving cost on an NVIDIA A10G, batch size 1, 256 new tokens, mean of three requests.

Mode	Tokens/s	Seconds/request	Prefill (s)	Peak memory (MiB)	Δ vs bf16 (s)
bf16	38.4	6.66	0.046	3183	—
8-bit	7.9	32.34	0.161	1936	−25.7
4-bit	29.6	8.66	0.052	1387	−2.0

6.7 End-to-end slash

We deployed the contract on a local Anvil chain (chain id 31337) with a mock ERC-20 stake of 100 tokens. A provider served the full fine-tune for prompt 3 and submitted its last-hidden proofs. The validator's check measured 57 exponent mismatches against the threshold of 46.02 and flagged the sequence; the per-layer classifier labelled it class 4 (fine-tune); the validator upheld the challenge. The provider's stake dropped to zero and its on-chain tamper history became `[4]`. This is a single demonstration on a prompt that the last-hidden check happens to catch; over 200 prompts that check catches the same variant 54.5% of the time. Per-layer feature extraction on the live path matches the offline path exactly (maximum absolute difference 0 on 10 sequences), because both call the same function.

7 Discussion and limitations

What the per-layer profile adds. Within a single model, GPU type and set of honest kernels, comparing top-k activations at every layer turns TOPLOC's binary check into a five-way attribution that also detects quantisation the last-layer check misses. The ablations locate the gain in the early and middle layers, where honest kernel noise and weight changes have different shapes.

What it would cost. Committing to 28 layers instead of one multiplies the proof by up to 28. TOPLOC reports 258 bytes per 32 tokens for one layer of an 8B model; a direct extension would be several kilobytes per 32 tokens. We have not built such a commitment: our per-layer comparison uses teacher-forced prefills on both sides, not decode-time proofs, so its accuracy under decode-versus-prefill noise is untested.

Limitations.

- **Scope.** One model (1.5B parameters), one main GPU type, four tamper variants of one strength each, 200 prompts per class. We have not tested larger models, other quantisation libraries (GPTQ, AWQ, FP8), weaker fine-tunes, or adaptive adversaries who train against the classifier.
- **Honest coverage.** The classifier rejects an honest kernel it was not trained on (Section 6.5), and even when trained on it, flags 6.5% of eager-kernel traffic.
- **Comparison with TOPLOC.** The last-hidden numbers come from decode-versus-prefill proofs and the per-layer numbers from prefill-versus-prefill comparisons, at different FPR budgets. The within-protocol ablation (Figure 5) is the controlled comparison.
- **Calibration.** 99th-percentile thresholds from 300 honest sequences have wide error bars; we report realised held-out FPR rather than the target.
- **Generation.** EOS is suppressed so that every sequence has eight checkpoints; pre-EOS controls are reported.
- **Cost.** Timing is for bitsandbytes kernels at batch size 1 on one GPU and does not establish that quantisation is slower in general.

- **Enforcement.** One trusted validator; a local chain rather than a public network; one demonstrated slash. Speculative decoding remains undetectable.
- **Cross-hardware.** 50 L4 sequences, and the proof check is L4 against itself.

8 Conclusion

Once its thresholds are set to accept realistic honest kernel variation, TOPLOC's last-hidden-state check does not detect 8-bit or 4-bit quantisation of Qwen2.5-1.5B-Instruct and detects light fine-tuning on about half of prompts. Comparing the same top-k statistics at every decoder layer detects all four modifications at 1% FPR and identifies which one occurred, with most of the information in the early and middle layers rather than the last one. The profile depends on the honest configurations it is trained on and fails on an unseen attention kernel, so a deployment must fix or declare its kernels. Attaching the label to an optimistic slashing contract records what was detected, not only that something was. On our hardware, quantised serving was slower than bf16, so we cannot justify the size of the stake by a time saving; the case for a stake rests on memory savings and on fine-tuned behaviour changes, which we leave to future work.

References

[1] Will Cai, Tianneng Shi, Xuandong Zhao, and Dawn Song. Are you getting what you pay for? auditing model substitution in LLM APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2504.04715.

[2] KD Conway, Cathie So, Xiaohang Yu, and Kartin Wong. opML: Optimistic machine learning on blockchain. *arXiv preprint arXiv:2401.17555*, 2024.

[3] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Zhi Zheng, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[6] Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.20247.

[7] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. SafetyNets: Verifiable execution of deep neural networks on an untrusted cloud. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab: Connectionism, 2025.

[9] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[10] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, 1998.

[11] Jack Min Ong et al. TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, 2025. arXiv:2501.16007.

[12] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[13] Haochen Sun, Jason Li, and Hongyang Zhang. zkLLM: Zero knowledge proofs for large language models. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024. arXiv:2404.16109.

[14] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. SVIP: Towards verifiable inference of open-source large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2410.22307.

[15] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford Alpaca: An instruction-following LLaMA model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[16] Jason Teutsch and Christian Reitwießner. A scalable verification solution for blockchains. *arXiv preprint arXiv:1908.04756*, 2019.

[17] Florian Tramèr and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware. In *International Conference on Learning Representations (ICLR)*, 2019.

A Additional results

Table 6: Five-class balanced accuracy (%) for every configuration, 5-fold prompt-grouped cross-validation. Pooled: three honest axes averaged per prompt (fixed before the run). Separate: one honest vector per axis.

Text	Checkpoints	LR, pooled	k-NN, pooled	LR, separate
Matched	all eight	99.5 (99.1–99.9)	98.8	99.3
Matched	pre-EOS	98.8 (97.4–100)	96.7	—
Shared	all eight	99.8 (99.5–100)	98.3	99.1
Shared	pre-EOS	98.8 (97.4–100)	97.3	—

B Reproducibility

- Model `Qwen/Qwen2.5-1.5B-Instruct`, revision `989aa7980e4cf806f80c7fef2b1adb7bc71aa306`; `toploc` 0.1.6; PyTorch 2.11.0 (CUDA 12.8); transformers 5.17.0; driver 595.71.05.

Table 7: Leave-one-honest-axis-out. The withheld axis is excluded from training in every fold and scored on its test-fold prompts. Matched text; shared text gives identical counts.

Withheld axis	Flagged	Predicted class counts				
		Clean	8-bit	4-bit	LoRA	Fine-tune
Eager attention	100%	0	18	182	0	0
Batch 4	0.5%	199	0	0	0	1
Batch 4 + padding	0.5%	199	0	0	0	1

- Seeds: prompt sampling and classifier 20260923; LoRA data order 202609231; SFT data order 202609232. Calibration prompts 0–99, validation prompts 100–199.
- top-k recorder: forward hooks on each decoder layer output; k=128 by absolute value; positions $|\text{prompt}| + 32c$. The live validator and the offline tables call the same feature function.
- Contract: Solidity 0.8.24, Foundry; deployment address, transaction hash and the recorded proof bytes are in the released result files.
- The original run's choices (thresholds, gate criteria, classifier settings, pooled clean class, class identifiers) were written down before the corresponding experiments. The separate-axis, leave-one-axis-out, ablation and ROC analyses were added afterwards, on the same stored activations, and are computed by a single CPU script.